Assignment 4.

Questions 1. Create a new React component called PlaylistCounter that uses the useReducer hook to manage a count state representing the number of songs in a playlist, with buttons to increment, decrement, and reset the count

Answer:-

PlaylistCounter.jsx

```jsx
import { useReducer } from "react";

const initialState = { count: 0, };

const reducer = (state, action) => {
    switch (action.type) {
        case "INCREMENT":
            return { count: state.count + 1 };

        case "DECREMENT":
            return { count: state.count - 1 };

        case "RESET":
            return initialState;

        default:
            return state;
    }
};

const PlaylistCounter = () => {
    const [state, dispatch] = useReducer(
        reducer,
        initialState
    );

    return (
        <div>
            <h1>🎵 Playlist Counter</h1>

            <h2>
                Number of Songs: {state.count}
            </h2>

            <button
                onClick={() =>
                    dispatch({ type: "INCREMENT" })
                }
```

```
            >
                Add Song
            </button>

            <button
                onClick={() =>
                    dispatch({ type: "DECREMENT" })
                }
            >
                Remove Song
            </button>

            <button
                onClick={() =>
                    dispatch({ type: "RESET" })
                }
            >
                Reset
            </button>
        </div>
    );
};

export default PlaylistCounter;
```

App.jsx

```
import PlaylistCounter from './Exercise4/PlaylistCounter'

const App = () => {
  return (
    <div>
      <PlaylistCounter />

    </div>
  )
}

export default App
```

Question 2. Modify your PlaylistCounter to disable the decrement button when the count is zero, so the playlist cannot have negative songs.

Hint: Add a condition in your reducer or on the button's disabled prop.

Answer:-

```jsx
import React, { useReducer } from "react";

const initialState = {
    count: 0,
};

const reducer = (state, action) => {
    switch (action.type) {
        case "INCREMENT":
            return { count: state.count + 1 };

        case "DECREMENT":
            return {
                count:
                    state.count > 0
                        ? state.count - 1
                        : 0,
            };

        case "RESET":
            return initialState;

        default:
            return state;
    }
};

const PlaylistCounter = () => {
    const [state, dispatch] = useReducer(
        reducer,
        initialState
    );

    return (
        <div>
            <h1>🎵 Playlist Counter</h1>

            <h2>
                Number of Songs: {state.count}
            </h2>

            <button
                onClick={() =>
                    dispatch({ type: "INCREMENT" })
                }
            >
                Add Song
            </button>
```

```
            <button
                onClick={() =>
                    dispatch({ type: "DECREMENT" })
                }
                disabled={state.count === 0}
            >
                Remove Song
            </button>

            <button
                onClick={() =>
                    dispatch({ type: "RESET" })
                }
            >
                Reset
            </button>
        </div>
    );
};

export default PlaylistCounter;
```

Question 3. Build a Flipkart-style cart item quantity manager using useReducer: create a CartItem component that displays the current quantity and has +, -, and Reset buttons to update the quantity state.

Answer:-

CartItem.jsx

```
import React, { useReducer } from "react";

const initialState = {
    quantity: 1,
};

const reducer = (state, action) => {
    switch (action.type) {
        case "INCREMENT":
            return {
                quantity: state.quantity + 1,
            };

        case "DECREMENT":
            return {
                quantity:
```

```
                    state.quantity > 1
                        ? state.quantity - 1
                        : 1,
            };

        case "RESET":
            return initialState;

        default:
            return state;
    }
};

const CartItem = () => {
    const [state, dispatch] = useReducer(
        reducer,
        initialState
    );

    return (
        <div
            style={{
                border: "1px solid #ddd",
                padding: "20px",
                width: "300px",
                borderRadius: "10px",
            }}
        >
            <h2>🖩 iPhone 15</h2>

            <p>Price: ₹79,999</p>

            <h3>Quantity: {state.quantity}</h3>

            <button
                onClick={() =>
                    dispatch({ type: "DECREMENT" })
                }
                disabled={state.quantity === 1}
            >
                -
            </button>

            <button
                onClick={() =>
                    dispatch({ type: "INCREMENT" })
                }
            >
```

```
                +
            </button>

            <button
                onClick={() =>
                    dispatch({ type: "RESET" })
                }
            >
                Reset
            </button>
        </div>
    );
};

export default CartItem;
```

App.jsx

```
import React from "react";
import CartItem from "./CartItem";

const App = () => {
  return (
    <div>
      <h1>🛒 Flipkart Cart</h1>

      <CartItem />
    </div>
  );
};

export default App;
```

Question 4. Given this code snippet using useState to manage a counter, refactor it to use useReducer instead:

`const [count, setCount] = useState(0);`

Make sure to implement increment, decrement, and reset actions.

Answer:-

useReducer.jsx

```
import React, { useReducer } from "react";

const initialState = 0;

const reducer = (state, action) => {
    switch (action.type) {
        case "INCREMENT":
```

```
            return state + 1;

        case "DECREMENT":
            return state - 1;

        case "RESET":
            return initialState;

        default:
            return state;
    }
};

const Counter = () => {
    const [count, dispatch] = useReducer(
        reducer,
        initialState
    );

    return (
        <div>
            <h1>Count: {count}</h1>

            <button
                onClick={() =>
                    dispatch({ type: "INCREMENT" })
                }
            >
                Increment
            </button>

            <button
                onClick={() =>
                    dispatch({ type: "DECREMENT" })
                }
            >
                Decrement
            </button>

            <button
                onClick={() =>
                    dispatch({ type: "RESET" })
                }
            >
                Reset
            </button>
        </div>
    );
```

```
};

export default Counter;
```